

Cahier des Charges & Spécifications Finales

Conception et développement d'une application Web d'envergure avec Symfony

THÉMATIQUE

Sujet Libre

COMPOSITION

Groupe de 1, 2 ou 3 personnes

TECHNOLOGIE

Symfony 6.x / 7.x & Twig

Objectif : Réaliser une application web professionnelle, robuste et sécurisée en appliquant l'ensemble des bonnes pratiques de l'écosystème Symfony. Vous devez faire preuve d'autonomie dans le choix de votre sujet tout en respectant scrupuleusement les exigences techniques ci-dessous.

1. Documentation & Livrables obligatoires

- **Cahier des charges complet :** Description fonctionnelle du projet, cas d'utilisation (Use Cases) et description des rôles utilisateurs.
- **Schéma de la base de données :** Modèle Conceptuel de Données (MCD) ou diagramme de classes UML clair et à jour.
- **Fixtures :** Un jeu de données de test complet et réaliste (utilisant DoctrineFixturesBundle et potentiellement Faker) permettant de tester l'application immédiatement.
- **Guide d'installation (README.md) :** Un document précis détaillant la procédure pour installer le projet en local, monter les conteneurs (si applicable), lancer les migrations, charger les fixtures, exécuter les tests, ainsi que la liste des identifiants des comptes de test pour chaque rôle.

2. Architecture des Données & Entités

Votre base de données doit refléter une logique métier complexe et respecter l'intégrité référentielle :

- **10 entités minimum :** Le modèle doit comporter au moins dix tables gérées via Doctrine.
- **Héritage d'entités :** Mise en œuvre obligatoire de l'héritage avec Doctrine (ex: Single Table Inheritance ou Class Table Inheritance) pour factoriser des comportements ou profils (ex: une entité parente User déclinée en plusieurs sous-types ou un tronc commun Product).

- **Relations complexes :**

- Au minimum **2 relations ManyToMany** (avec ou sans attributs de liaison).
- Au minimum **8 relations OneToMany / ManyToOne** bien modélisées.

3. Sécurité & Droits d'Accès

- **Authentification sécurisée :** Utilisation du Security Component de Symfony, avec hachage des mots de passe (Password Hasher) et formulaire de connexion/déconnexion natif.
- **Hierarchie des rôles :** Définition claire de minimum **3 rôles différents** (ex: ROLE_USER, ROLE_MANAGER, ROLE_ADMIN) cloisonnant les fonctionnalités de l'application.
- **Voter personnalisé :** Implémentation d'au moins **1 Voter** sur mesure pour gérer des permissions fines et dynamiques liées au cycle de vie des objets (ex: droit de modifier une ressource uniquement si l'utilisateur en est l'auteur).

4. Composants API & Communications

- **Contrôleur API dédié :** Création d'un endpoint API structuré (ex: /api/v1/...) retournant du flux JSON, en exploitant le Serializer de Symfony (gestion fine du Context, des Groupes de normalisation et de dénormalisation).
- **Envoi de courriels :** Intégration du composant Mailer / Notifier pour l'envoi d'e-mails transactionnels asynchrones ou événementiels (ex: confirmation d'inscription, notification de commande).
- **Consommation d'une API externe :** Connexion de votre application à un service tiers via le HttpClient de Symfony. Exemples suggérés : intégration d'une IA (OpenAI), passerelle SMS (Twilio), API de géolocalisation, météo, ou services financiers.

5. Fonctionnalités Avancées & Qualité du Code

- **Interface d'administration :** Un espace d'administration dédié et sécurisé pour la gestion globale des données (CRUD avancé, statistiques), développé sur mesure en Twig ou via EasyAdminBundle.
- **Formulaires dynamiques :** Utilisation de formulaires complexes avec l'utilisation d'événements de formulaire (Form Events : PRE_SET_DATA, PRE_SUBMIT) pour modifier les champs dynamiquement en fonction des choix de l'utilisateur.
- **Requêtes personnalisées :** Implémentation de méthodes spécifiques dans vos Repositories à l'aide du QueryBuilder pour optimiser les performances (jointures indispensables pour éviter le problème du N+1) et faire des filtres avancés.
- **Interface utilisateur (Twig) :** L'application doit posséder au moins **10 pages distinctes** générées via Twig, intégrant l'héritage de templates (système de blocs), des filtres personnalisés et une UX soignée.

- **Stratégie de tests** : Écriture obligatoire d'au moins **1 test unitaire** (ex: test d'un service ou d'une entité) et d'au moins **1 test fonctionnel** (ex: scénario de connexion ou soumission de formulaire) avec WebTestCase.

6. Intégration & Déploiement Continu (CI/CD)

- **Pipeline CI fonctionnel** : Automatisation de la qualité à chaque push (via GitHub Actions, GitLab CI, etc.) comprenant :
 - Le linter Symfony (validation des fichiers de config, conteneur et templates Twig).
 - L'analyse statique du code via **PHPStan** (niveau 5 minimum recommandé).
 - L'exécution automatique de votre suite de tests.
- **Déploiement** : L'application doit être hébergée et accessible en ligne (ex: Platform.sh, Heroku, Render, VPS ou équivalent).

7. Barème & Points Bonus

Pour valoriser les architectures poussées et l'exploration technique, les implémentations suivantes rapporteront des points bonus significatifs :

TECHNOLOGIE / CONCEPT	DESCRIPTION DE LA MISE EN ŒUVRE	TYPE
Temps Réel	Utilisation du composant Mercure pour pousser des mises à jour instantanées sur le front-end.	BONUS
Asynchronisme	Mise en place de files d'attente avec le composant Messenger (transport AMQP, Doctrine, ou Redis) pour le traitement des tâches lourdes.	BONUS
Commandes CLI	Création de commandes de console personnalisées (Console Command) utiles pour la maintenance ou l'administration.	BONUS
Tests de Mutation	Validation de la robustesse des tests via un outil comme Infection PHP.	BONUS
DDD / TDD	Développement guidé par les tests (TDD) ou découpage architectural orienté Domain-Driven Design (DDD).	BONUS